Multi-Stage Retrieval & Ranking for Retail

A Two-Tower Recommender with RAG-Based Compliance Agents

Rishab | Data Scientist

Home Depot — Technical Presentation

github.com/Rishab-KH/multi-stage-retrieval-ranking-recommendation

3.4×
Recall lift

2.8×
NDCG lift

6
Pipeline stages

BUSINESS PROBLEM

From Relevance Models to Retail Agents

Modern retail doesn't just need recommendations — it needs intelligent agents that can check inventory, enforce policies, negotiate with vendors, and explain their decisions. This project builds that full stack.

Recommendation Quality

Most recsys optimize for relevance but ignore inventory, policy, and explainability constraints

Inventory Blindness

Recommending OOS or low-stock items erodes trust, increases returns, and wastes fulfillment resources

No Compliance Trail

Without grounded explanations, compliance teams can't verify why a product was recommended or substituted

Agent-Ready Patterns

The same retrieval + reasoning + structured output pipeline applies to vendor negotiation and warehouse agents

ARCHITECTURE

End-to-End Pipeline

Six stages: two-stage retrieval + ranking → inventory constraints → RAG policy retrieval → grounded generation. Orchestrated by LangGraph.

1

Two-Tower Retrieval

FAISS ANN → top-200

→

2

GBM Reranker

6 features → top-10

→

3

Inventory Constraints

OOS filter + substitution

→

4

Policy Retrieval

Hybrid dense+BM25+RRF

→

5

Grounded Generation

Cited JSON audit

ML Backbone (Stages 1–3)

Agent Layer (Stages 4–5)

Tech: PyTorch, FAISS, LightGBM, LangGraph, OpenAI, Cohere, MLflow, DeepEval

ARCHITECTURE

End-to-End System Architecture

STAGE 1 — Recommendation Engine

User ID

Two-Tower Model (MLP towers)

FAISS IndexFlatIP ⟶ Top 200

LightGBM LambdaRank (6 features)

Top-10 Recommendations

→

STAGE 2 — RAG Policy Agent (LangGraph)

Inventory Constraints (stock + subs)

Policy Router ⟶ doc routing

Hybrid Retrieval (FAISS+BM25+RRF)

Cohere Reranker ⟶ Top-6 chunks

GPT-4o Answer + Citations

Evaluation: Recall@K, NDCG@K, MRR@K | DeepEval: Faithfulness, Hallucination, Compliance | MLflow Tracking

DATA

Dataset & Evaluation Setup

3.4M+	206K	49.7K	21
Orders	Users	Products	Departments

Instacart Open-Source Dataset

- Implicit purchase interactions (orders × products)
- Side features: aisle_id, department_id per product
- Filtered to users with 3+ orders for signal quality

Temporal Train/Val/Test Split

- Train: all orders except last two per user
- Validation: second-to-last order (early stopping)
- Test: last order (final held-out evaluation)
- No random splits — prevents future data leakage

Policy corpus: 10 Markdown documents, semantic section-chunked (≤ 1200 chars), covering substitutions, cold chain, delivery windows, bulk limits, and promos — simulating a real retail ops library.

MODEL

Two-Stage Retrieval & Ranking

Stage 1: Two-Tower Neural Retrieval

User Tower

```
user_id → Emb(128)
Linear→LN→GELU→Drop(
.15)
Linear→LN→GELU→Drop(
.1)
Linear(256→128) +
residual
L2 normalize
```

Item Tower

```
item+aisle+dept → 176
Linear→LN→GELU→Drop(
.15)
Linear→LN→GELU→Drop(
.1)
Linear(256→128) + proj
res
L2 normalize
```

Loss: InfoNCE with in-batch negatives + hard negatives

Debiasing: Popularity sampling-bias correction (logit -= log q_j)

Inference: FAISS IndexFlatIP → top-200 candidates per user

Stage 2: GBM Reranker

Gradient Boosting classifier with CalibratedClassifierCV (sigmoid), trained on validation users, evaluated on held-out test users.

6 Hand-Crafted Features

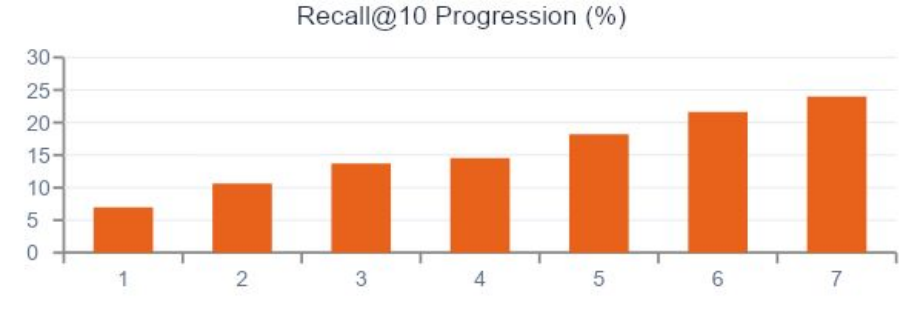
Feature	Signal
Similarity score	Two-tower dot product
log(popularity)	Global purchase frequency
History flag	User purchased before?
Item reorder rate	Repeat-purchase intent
log(user orders)	User activity level
log(user history)	Basket breadth

Critical fix: original code trained reranker on test ground truth (label leakage). Now uses val users for training, test users for final eval.

RESULTS

Performance: Baseline → Final System

Experiment	Recall@10	NDCG@10	Recall@20	NDCG@20	Lift
Popularity baseline	0.0699	0.0976	0.0955	0.0974	—
Two-Tower v1 (embed only)	0.1065	0.1047	0.1427	0.1140	+52%
GBM reranker (3 feat, k=20)	0.1369	0.1423	0.1427	0.1326	+96%
Two-Tower v2 (MLP + content)	0.1451	0.1742	0.2073	0.1854	+108%
GBM reranker v2 (6 feat, k=200)	0.1819	0.2315	0.2073	0.2179	+160%
LambdaRank LTR reranker	0.2160	0.2543	0.2850	0.2639	+209%
Deep TT v2 + LTR (3L, GELU, res)	**0.2400**	**0.2751**	**0.3268**	**0.2919**	**+243%**



Overall Improvement

Recall@10	0.0699	→	0.2400	**+243%**
NDCG@10	0.0976	→	0.2751	**+182%**
Recall@20	0.0955	→	0.3268	**+242%**

ONLINE METRICS

Online Metrics & Customer Funnel

Offline metrics (Recall, NDCG) validate model quality. Online metrics measure real business impact at each stage of the customer journey.

Browse / Homepage

Recommendations

"Recommended for you"
"Trending in your area"

Key Metrics

CTR
Impression-to-click rate
Session depth

Product Page

Recommendations

"Frequently bought together"
"Similar items"

Key Metrics

Add-to-cart rate
Cross-sell attach rate
PDP bounce rate

Cart / Checkout

Recommendations

"You might also need"
Substitutions for OOS

Key Metrics

Cart addition rate
AOV lift ($)
Substitution accept rate

Post-Purchase

Recommendations

"Reorder your favorites"
"Based on last order"

Key Metrics

Reorder rate
30-day retention
Lifetime value (LTV)

Offline: Recall@K, NDCG → model selection & iteration

Online: CTR, Add-to-Cart, AOV, LTV → business impact

ABLATION

What Drove Each Gain

Model Architecture

3-layer MLP + residual

Deeper towers with skip connections — preserves embedding identity while learning non-linear transforms

LayerNorm + GELU

Stabilises training with hard negatives; GELU's smooth activation outperforms ReLU in retrieval

Content embeddings (32+16)

Aisle + department features solve cold-start — new items share structure with known catalog

Kaiming init + Dropout 0.15/0.1

Correct init for deep towers; graduated dropout prevents co-adaptation in deeper layers

Training Quality

Hard negative mining

35% user sample per epoch (up from 15%) — richer adversarial signal, better near-miss separation

Popularity debiasing

logit -= log(q_j) corrects over-penalizing popular items as in-batch negatives

Temporal validation split

Prevents future data leakage — accurate early-stopping signal

Patience 2→4

Harder training objective = noisier loss curves; more patience avoids premature stopping

Retrieval Pipeline

Candidate pool 20→200

Reranker can only recover what retrieval found — wider pool raises recall ceiling dramatically

6 features (from 3)

Reorder rate + user activity features give the GBM stronger purchase-intent signals

Label leakage fix

Reranker now trains on val users, not test — honest eval on unseen users

LambdaRank objective

Optimizes NDCG directly — position-aware gradients vs pointwise loss (+209% total)

CONSTRAINTS

Inventory-Aware Filtering & Substitution

Candidate Splitting

Top-10 reranked items become primary recommendations. Positions 11–50 form the substitute candidate pool.

Substitutes never drawn from the primary set — maintains recommendation integrity.

→

Stock Status Check

Each product gets a simulated stock flag based on popularity percentile and reorder rate.

- **In Stock**
- **Low Stock**
- **Out of Stock**

→

Substitution Logic

OOS items replaced by best substitute from candidate pool:

1. **Same aisle + dept**
2. **Same dept, diff aisle**
3. **Any available product**

Warnings ⟶ consumed by RAG layer downstream.

RAG AGENT

Hybrid Policy Retrieval Pipeline

Multi-signal query construction feeds parallel dense + sparse retrieval, fused via Reciprocal Rank Fusion, with deterministic policy routing.

Query Construction

`intent | departments | aisles | warnings | substitute names | top product names → composite query`

Dense FAISS

text-embedding-3-small → IndexFlatIP → top-80

Sparse BM25

Regex tokenizer → BM25Okapi → top-80

RRF Fusion (k=60)

`score(d) = Σ 1/(k + rank) across both retrievers`

Cohere Reranker (optional)

`Cross-encoder rerank-english-v3.0 → top-6 chunks`

GENERATION

Policy-Grounded LLM Generation

The LLM acts as a compliance auditor — it cannot modify recommendations, only explain them against retrieved policy evidence.

Decision Priority

1. **Inventory Constraints**
   Hard filter — OOS never recommended
2. **Policy Compliance**
   Cold chain, bulk limits, substitution rules
3. **Recommendation Score**
   Model relevance — lowest priority

Output: Structured JSON — per-item SKU, inventory_status, reason, policy_citations, policy_notes. Machine-parseable for downstream systems.

Built-in Guardrails

- Prompt injection defense — policy docs treated as untrusted data
- Citation integrity — every claim must trace to a retrieved source
- Post-gen claim verification — lexical overlap against chunks
- Inventory lock — LLM cannot upgrade stock status of any item
- Structured JSON output — parseable, machine-readable per-item audit

EVALUATION

RAG Pipeline: Live Demo Results

Pipeline Telemetry	User 6 — perishable delivery	User 2 — bulk staples	User 10 — organic subs
Latency (total)	58.0s (cold*)	37.5s	**28.2s**
Retrieval score (avg / max)	**0.844 / 0.947**	**0.758 / 0.953**	0.658 / 0.997
Cost per run	$0.042	$0.043	$0.038
Citations / Chunks	produce + delivery + bulk / 5	dairy + bulk + promo / 3	produce / 2

DeepEval LLM-as-Judge Quality Metrics (7 demo users)

User	Intent	Faith ↑	Halluc ↓	Ctx Recall ↑	Compliance ↓
6	Perishable delivery	**1.00**	0.33	0.85	0.28
2	Bulk staples + promo	0.80	0.83	0.78	0.48
10	Organic substitutions	0.95	0.70	0.81	0.42
1	Snack variety + refund	**1.00**	0.40	0.77	0.44
17	Frozen meal + cold chain	**1.00**	0.33	**0.99**	0.31
4	Household restock	0.90	0.83	0.74	0.59
9	Dairy + sub flexibility	0.83	**0.17**	0.84	**0.27**
	Average (7 users)	**0.93**	**0.51**	**0.83**	**0.40**

APPLICABILITY

How This Maps to Home Depot Agents

The same multi-stage retrieval + grounded reasoning pattern directly applies to vendor negotiation and warehouse agents.

This Project		Home Depot Agent
Product catalog retrieval (FAISS)	→	Vendor catalog + warehouse inventory lookup
Inventory constraint checking	→	Real-time stock across 2,300+ stores
Policy document retrieval (RAG)	→	Vendor contracts, pricing rules, bulk terms
Substitution logic (OOS → alt)	→	Alternative vendor/SKU sourcing
Grounded JSON output + citations	→	Auditable negotiation proposals with contract refs
LLM guardrails + prompt injection defense	→	Safe agent behavior with untrusted vendor data
LangGraph orchestration	→	Multi-step agent workflows with state management

Key insight: The hard part of building retail agents isn't the LLM — it's the retrieval pipeline, constraint enforcement, and output validation. This project builds all three.

COMMUNICATION

Communicating Results to Stakeholders

Technical wins only matter if they translate to business outcomes stakeholders can act on. Each audience needs a different framing.

Merchandising & Product

What they care about:

- Add-to-cart rate uplift
- Average order value (AOV) impact
- Cross-sell & attachment rates

How I'd frame it:

"Our improved recommendations surface products customers actually want — 160% more accurate at predicting next purchase. This means higher cart conversion and larger baskets."

Operations & Supply Chain

What they care about:

- OOS recommendation rate reduction
- Substitution acceptance rate
- Fulfillment efficiency (fewer exceptions)

How I'd frame it:

"We no longer recommend products we can't deliver. The system checks stock in real-time and offers smart substitutes from the same aisle — reducing fulfillment exceptions and customer complaints."

Executive / Finance

What they care about:

- Revenue per session lift
- Customer retention & LTV
- Compliance risk reduction

How I'd frame it:

"Every recommendation now comes with an auditable compliance report. This protects us legally and builds customer trust — driving retention and lifetime value."

Principle: translate model metrics → funnel metrics → business KPIs. Never present Recall@K to a non-technical audience.

REFLECTION

Challenges & Personal Growth

Technical Challenges

Label Leakage in Reranker

Original code trained the GBM on test set ground truth — inflating reported metrics. Fixed by training on validation users and evaluating on held-out test users.

Hard Negative Mining Cost

Mining hard negatives every epoch is expensive. Solved by sampling 35% of users per epoch — maintains gradient quality while keeping training feasible.

Intent vs. Recommendations

The retriever doesn't see user intent — same items regardless of query. Addressed by making the RAG layer an auditor; intent-aware re-ranking is planned future work.

BM25 Tokenizer Bug

Naive whitespace splitting treated 'delivery,' and 'delivery' as different tokens. Fixed with regex tokenizer — required rebuilding the sparse index.

What I Learned

- Two-stage retrieval + ranking is industry standard for a reason — separation of concerns enables independent optimization
- Evaluation rigor matters more than model complexity — fixing label leakage mattered more than architecture changes
- Hybrid retrieval (dense + sparse + RRF) catches different failure modes than either alone
- LLM guardrails for production — prompt injection defense, output validation, claim verification
- Design honesty — knowing when the RAG layer is explaining vs. deciding, and being clear about both

PRODUCTION

Scaling to Production on GCP

Every component in the current pipeline maps to a managed GCP service — removing operational overhead and enabling scale.

Current Component	GCP Service	Why
Training data (CSV files)	**BigQuery**	SQL-native feature engineering, scheduled queries for train/val/test splits, BQ ML for baselines
Feature computation (Pandas)	**BigQuery + Vertex AI Feature Store**	Point-in-time lookups, real-time serving of user/item features to reranker
Two-tower training (PyTorch)	**Vertex AI Training (GPU)**	Managed training jobs, hyperparameter tuning, experiment tracking
FAISS ANN index	**Vertex AI Vector Search**	Managed approximate nearest neighbor, auto-scaling, low-latency serving
GBM reranker serving	**Vertex AI Endpoints**	Online prediction with autoscaling, A/B traffic splitting for experiments
Inventory simulation	**BigQuery (live inventory table)**	Real-time stock from warehouse feeds, streamed via Pub/Sub → BigQuery
Policy RAG index	**Vertex AI Search**	Managed retrieval with grounding, automatic chunking and re-indexing
LLM generation (OpenAI)	**Gemini via Vertex AI**	Data residency compliance, grounding API, lower latency for GCP-native stack
ML flow tracking	**Vertex AI Experiments**	Native experiment tracking, model registry, lineage with BigQuery datasets

BigQuery as the central hub: training data, feature engineering, inventory feeds, and evaluation metrics all live in BQ — enabling SQL-native analysis, scheduled retraining pipelines, and direct integration with Vertex AI.

Future Work & Next Steps

Intent-aware re-ranking

Let retrieved policies influence which candidates make the top-10, not just explain them

Real inventory API

Replace simulated stock flags with live warehouse/store inventory feeds

Vendor negotiation agent

Extend the same pipeline to query vendor contracts and propose optimal pricing

Online A/B testing

Measure recommendation + agent quality in production with controlled experiments

---

Thank You

Looking forward to your questions.